CNN PROJECT • SUBMISSION 2 OF 4

Pretrained Models, With and Without Fine-Tuning

Three ImageNet backbones, two transfer-learning modes, six runs

Repository: github.com/Rayhan-who/cricket-shot-attention-cnn

Summary

Six training runs were conducted: ResNet50, EfficientNetB0 and DenseNet121, each evaluated as a frozen feature extractor and as a fine-tuned network, for up to 40 epochs with early stopping. All six completed in 80.5 minutes on a Kaggle T4 GPU.

The best configuration, EfficientNetB0 fine-tuned, reached **0.9972** test accuracy and **0.9973** macro F1 — 719 of 721 test images correct. Fine-tuning improved macro F1 for all three architectures, by a mean of +0.0478. No run showed overfitting.

> **These absolute accuracies are almost certainly inflated by residual duplicate leakage in the dataset, and should not be reported without the qualification set out in Section 4.** The relative findings — the fine-tuning effect, the per-class ordering and the cost measurements — are unaffected.

1 Experimental setup

1.1 Inherited from Submission 1

The following were fixed in Submission 1 and are not re-decided here. Holding them constant is what makes the six runs comparable with each other and with Submissions 3 and 4.

Setting	Value
Split	70 / 15 / 15, grouped by duplicate identity, stratified by class
Split sizes	train 3,321 • validation 682 • test 721
Random seed	42
Channel conversion	forced RGB (4,586 of 4,724 images are RGBA)
Input resolution	224×224
Augmentation	rotation ±10°, zoom 0.8–1.0, brightness ±20%, contrast ±20%
Horizontal flip	**not applied** — mirroring inverts the label on leg-side strokes
Class weights	not applied — class imbalance is 1.12 : 1

The split was not transferred as a file. It was *recomputed* from the same deterministic procedure and then verified against Submission 1's counts; the notebook asserts the match and reports "Split reproduces Submission 1 exactly". An assertion also confirms that no duplicate group spans a partition boundary.

1.2 What varies

Factor	Levels
Backbone	ResNet50, EfficientNetB0, DenseNet121 (ImageNet weights)
Transfer mode	frozen (head only) • fine-tuned (whole backbone)
Learning rate	10^{-3} frozen, 10^{-4} fine-tuned
Epoch budget	40, with early stopping on validation loss, patience 8

Two implementation decisions worth recording

Each backbone's own `preprocess_input` is applied inside the model. ResNet50 expects Caffe-style mean subtraction, DenseNet Torch-style normalisation, and EfficientNet raw $[0, 255]$. Placing the preprocessing inside each model lets all six runs share one set of `uint8` arrays with no possibility of applying the wrong normalisation — an error that would be silent and would look like a modelling result.

BatchNorm is held in inference mode throughout, including when fine-tuning. The backbone is always called with `training=False`. Updating BatchNorm statistics while fine-tuning on a small dataset destabilises training and is a common cause of fine-tuned models scoring *below* frozen ones.

2 Results

2.1 All six runs

Backbone	Mode	Acc.	Prec.	Rec.	Macro F1	Ep.	Train	ms/img
EfficientNetB0	fine-tuned	**0.9972**	0.9973	0.9973	**0.9973**	24	10.6 m	8.73
DenseNet121	fine-tuned	0.9972	0.9972	0.9973	0.9972	24	17.5 m	14.87
ResNet50	fine-tuned	0.9945	0.9945	0.9945	0.9944	29	19.5 m	9.75
ResNet50	frozen	0.9750	0.9754	0.9752	0.9752	40	12.0 m	9.72
EfficientNetB0	frozen	0.9598	0.9610	0.9599	0.9602	40	**6.6 m**	9.18
DenseNet121	frozen	0.9098	0.9122	0.9105	0.9103	40	13.3 m	16.47

Precision and recall are macro-averaged. "Ep." is the epoch at which training stopped out of a budget of 40.

2.2 The effect of fine-tuning

Backbone	Frozen	Fine-tuned	Gain
DenseNet121	0.9103	0.9972	**+0.0870**
EfficientNetB0	0.9602	0.9973	+0.0371
ResNet50	0.9752	0.9944	+0.0192
Mean			**+0.0478**

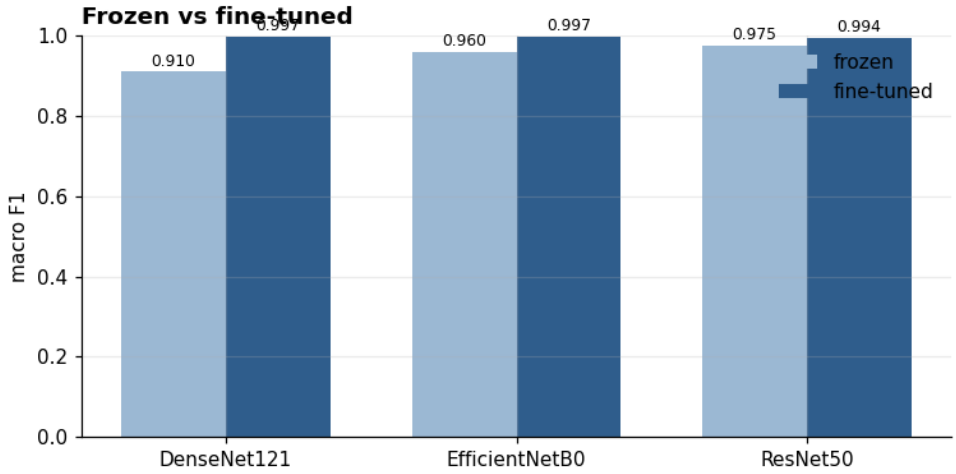


Figure 1: Macro F1 for each backbone, frozen versus fine-tuned. The direction is the same in all three cases; the magnitude is largest where the frozen baseline was weakest.

The gain is inversely related to the frozen baseline, which is the expected pattern: fine-tuning contributes most where the frozen ImageNet representation transfers least well to the target domain.

A second signal supports the same reading. **All three frozen runs consumed the full 40-epoch budget without triggering early stopping**, while all three fine-tuned runs converged and stopped early — at 24, 24 and 29 epochs. The frozen models were still improving when the budget expired; the fine-tuned models had finished.

2.3 Training curves

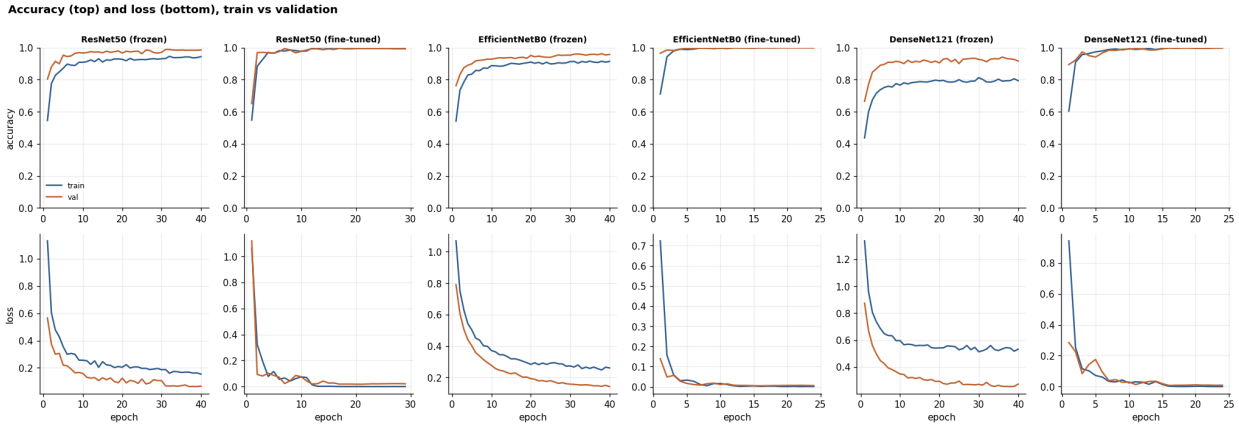


Figure 2: Accuracy (top row) and loss (bottom row), training versus validation, for each of the six runs. Note that the validation curve sits *above* the training curve in the frozen runs: training accuracy is measured on augmented images and validation on clean ones, so this ordering is the expected healthy state under this augmentation policy, not a sign of underfitting.

2.4 Confusion matrices

Submission 1 predicted, from visual inspection of the sample grid, that `drive` and `legglance-flick` would be the hardest pair to separate. The automated check reports `drive` ↔ `pullshot` instead, so the **prediction was not confirmed**.

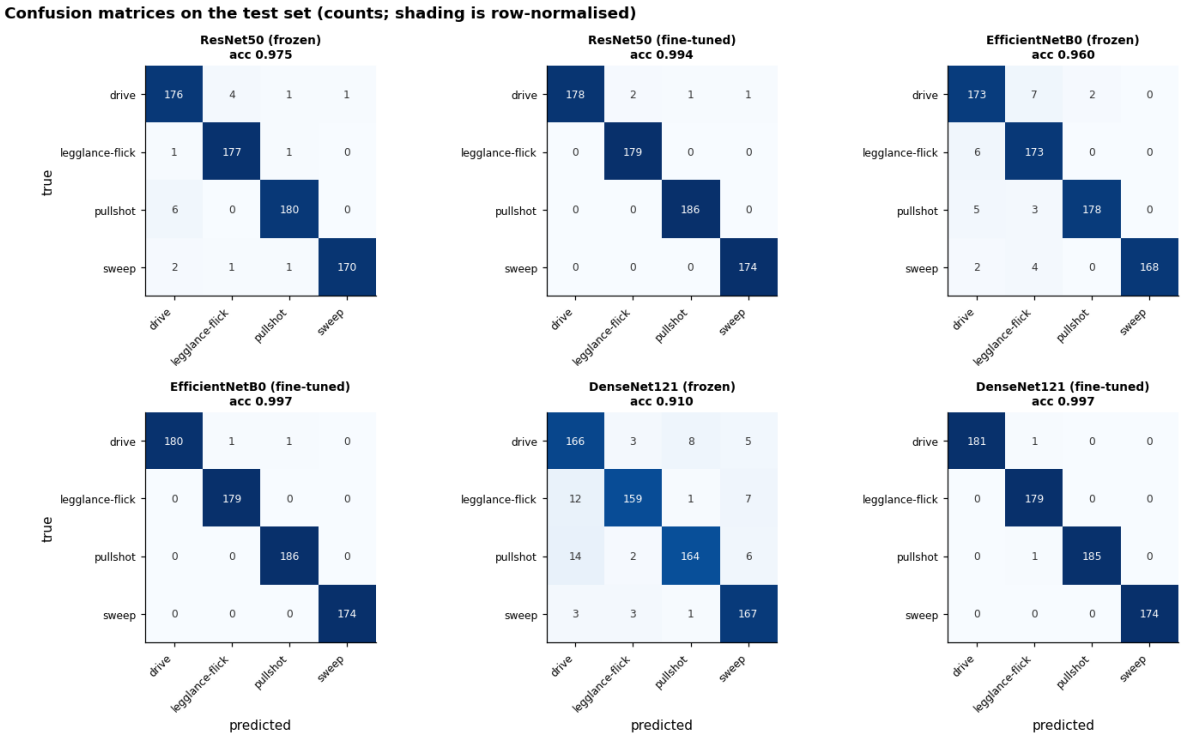


Figure 3: Test-set confusion matrices for all six runs. Cells show counts; shading is row-normalised.

That result should not be over-read. The best model makes **two errors in total**, so the "hardest pair" is being inferred from a single misclassification. At this accuracy the confusion matrix carries almost no information. The more reliable signal comes from the weakest run — DenseNet121 frozen — where `drive` has the lowest per-class F1 by a clear margin.

2.5 Per-class performance

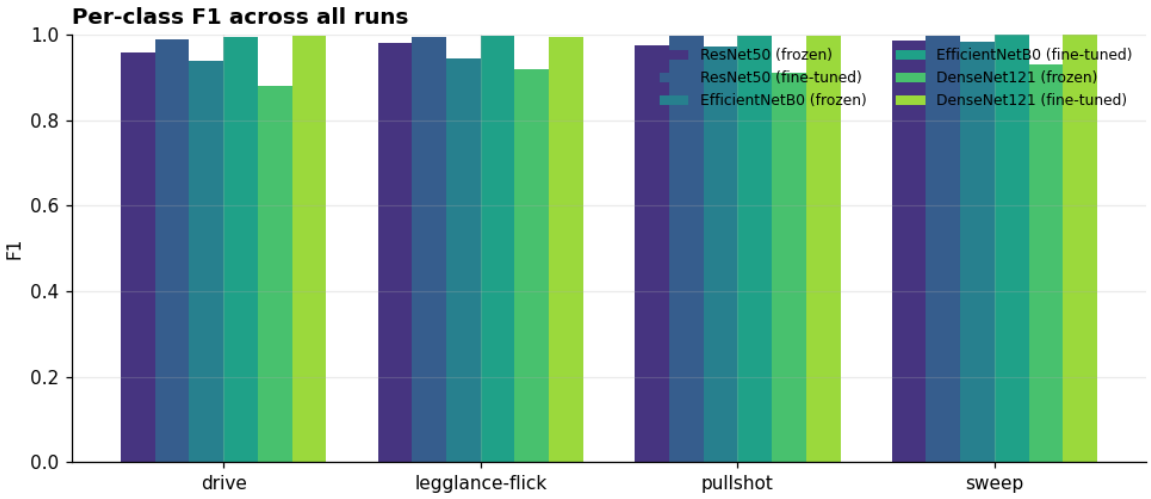


Figure 4: Per-class F1 across all six runs.

Class	Mean F1 across runs	
`drive`	0.9601	hardest
`legglance-flick`	0.9719	
`pullshot`	0.9752	
`sweep`	0.9826	easiest

The ordering is stable: it holds in every one of the six runs. `sweep` is easiest throughout, which is consistent with it being the most visually distinct posture — it is the only stroke played from a lowered or kneeling position, so its silhouette differs from the other three in a way that survives heavy downsampling. `drive` is hardest throughout, which is consistent with it being an upright side-on stroke that shares its body geometry with both `legglance-flick` and `pullshot`.

2.6 Cost

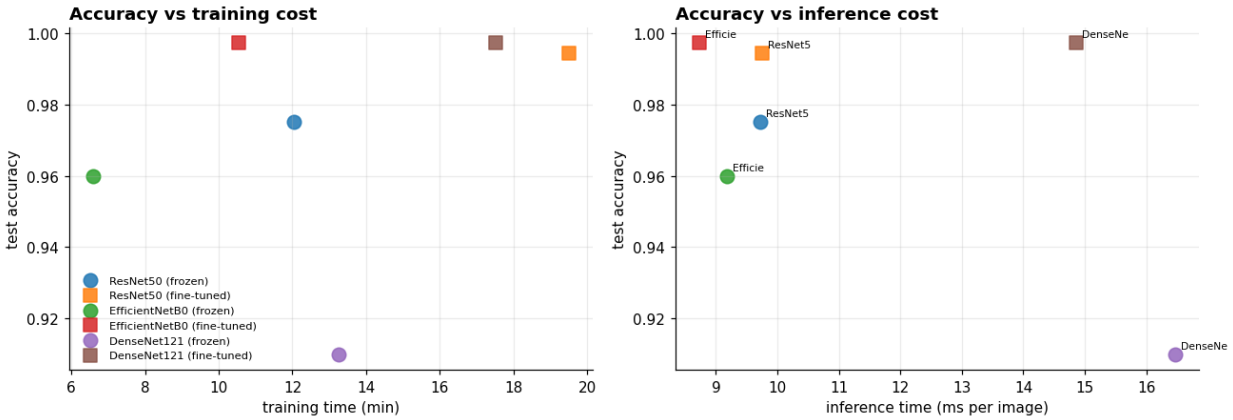


Figure 5: Test accuracy against training time (left) and against inference time (right). Circles are frozen runs, squares fine-tuned.

Training ranged from 6.6 minutes (EfficientNetB0 frozen) to 19.5 minutes (ResNet50 fine-tuned). Inference ranged from 8.73 to 16.47 ms per image.

EfficientNetB0 fine-tuned is the best configuration on every axis that matters. It has the highest accuracy, the fastest inference (8.73 ms/image), the second-fastest training (10.6 min), and the fewest trainable parameters among the fine-tuned runs (3.97 M, against 23.49 M for ResNet50). DenseNet121 matches its accuracy but costs 66% more training time and 70% more inference time for no measurable benefit.

3 Overfitting and underfitting

Backbone	Mode	Train–val gap	Assessment
ResNet50	fine-tuned	+0.0070	healthy
DenseNet121	fine-tuned	+0.0029	healthy
EfficientNetB0	fine-tuned	+0.0012	healthy
ResNet50	frozen	−0.0416	healthy
EfficientNetB0	frozen	−0.0433	healthy
DenseNet121	frozen	−0.1224	healthy

Zero of six runs exceeded a train–validation accuracy gap of 0.15. No corrective action was required.

The negative gaps on the frozen runs are expected rather than a warning. Training accuracy is computed on augmented images while validation uses clean ones, so under this augmentation policy the validation score sitting above the training score is the normal state. The largest negative value, −0.1224 for DenseNet121 frozen, belongs to the weakest model overall and reflects a model still improving at the epoch budget rather than one that has failed to fit.

The regularisation in place throughout was: early stopping on validation loss with patience 8 and best-weight restoration; `ReduceLROnPlateau` at factor 0.3; dropout 0.3 before the classifier; rotation, zoom, brightness and contrast augmentation; and BatchNorm held in inference mode.

4 A caveat on these numbers

> **The absolute accuracies reported above are almost certainly inflated, and the inflation is a property of the dataset rather than of the models.**

The dataset author states on the Kaggle page that the images were collected from the internet and then augmented to produce the released collection. Submission 1 controlled for this by grouping images with identical perceptual hashes and splitting on groups rather than on files. That removed 608 redundant copies (12.9%) from consideration and prevented an estimated 17.9% of the test set from carrying an exact twin in training.

That control is incomplete. A difference hash is sensitive to geometric change: a rotated, flipped or cropped copy of a photograph produces a completely different hash and was therefore treated as an independent image. If the author's augmentation included any geometric transformation — and the phrase "augmented" usually implies it — those variants remain distributed across the train and test partitions, and the models are being scored partly on photographs they have already seen.

Two observations support this reading.

1. **DenseNet121 frozen reached 91.0% with 4,096 trainable parameters.** That is a linear classifier on fixed ImageNet features. It cannot learn anything cricket-specific. For it to reach 91% on a fine-grained four-class problem, test images must lie very close to training images in feature space.
2. **99.7% on a task characterised in Submission 1 as fine-grained.** All four classes share

one batsman, one pitch, comparable kit and comparable backdrops, with the label carried by posture and bat angle across a minority of the frame.

Neither observation is proof. The residual redundancy could genuinely be small and the task genuinely easy under ImageNet features. Resolving the question requires embedding-based similarity clustering rather than hashing, because rotated copies remain close in feature space even when their hashes diverge. That audit is the immediate next piece of work.

4.1 What this does and does not affect

Unaffected	Not trustworthy
The fine-tuning effect — the direction held in all three architectures	Absolute accuracy and macro F1
Per-class ordering — stable across all six runs	The confusion-matrix analysis at the top of the range
Training and inference timings	Use of 0.9973 as a target for Submission 3

4.2 A second limitation, unquantifiable with this dataset

Beyond duplicate leakage there is **session correlation**. Photographs taken at the same match share stadium, kit, lighting and crowd, so a model can learn the match rather than the shot even when no two images are duplicates. The correct remedy is a session-wise split, but this dataset ships no match metadata, and the filenames were renumbered into contiguous sequences after augmentation, destroying any provenance that might have allowed grouping. This is recorded as a known threat to validity rather than addressed.

5 Carrying into Submission 3

Item	Value
Split	loaded from `artifacts/split_manifest.csv` — not regenerated
Seed	42
Input, augmentation, metrics	unchanged
Best pretrained macro F1	0.9973 (EfficientNetB0 fine-tuned)

Submission 3 differs in exactly one respect: the network is trained from random initialisation rather than ImageNet weights. A from-scratch CNN is expected to score below the pretrained results — roughly 3,300 training images is not enough to learn general visual features from scratch — and that gap is the measured value of transfer learning.

The figure of 0.9973 should be treated as provisional. If the leakage audit shows substantial residual redundancy, this baseline and the Submission 3 result will both move downwards, and the comparison that matters is the one made on a corrected split. That also determines whether Submission 4 is viable: an attention module cannot demonstrate a measurable gain against a baseline already above roughly 96%, because the improvement required to clear run-to-run variance would exceed the headroom available.